

Queues and Log Files

Azure Queue Storage and Azure Files in the ABC Retail order processing platform

LIVE APPLICATION

<https://abcretail-gm-cldv7112.azurewebsites.net>

PUBLIC GITHUB REPOSITORY

<https://github.com/geniustaku/abc-retail-storage-console>

ACCESS

Open. The application requires no sign in, so no credentials are needed to review any page, place an order, or process a message off a queue.

STUDENT NAME

Genius Mhirizhonga

STUDENT NUMBER

ST10541838

SEMESTER

Semester 2, 2026

ASSESSMENT

ICE Task 2

MODULE CODE

CLDV7112w

DUE DATE

17 August 2026

SECTION 1

Brief and how it was met

Even after moving to the cloud, ABC Retail's order processing still leaned on custom pipelines and third-party messaging that delivered inconsistently and could not be scaled. This task replaces that with Azure Queue Storage for the work itself and an Azure file share for the operational record of what happened.

REQUIREMENT	HOW IT IS SATISFIED
Details relating to processing of orders stored in Azure Queues	Placing an order puts a message on the <code>order-processing</code> queue, for example <i>Processing order ORD-20260821-B11D for Michael Chen, 2 x Canvas Weekender Bag</i> . The order is not complete until that message is handled.
Details relating to inventory management stored in Azure Queues	Catalogue and stock movements go to <code>inventory-management</code> , including <i>Uploading image "7d727e6c...jpg", Inventory item created, Stock adjusted and Inventory item removed</i> .
Log files stored using Azure Files	Every action is appended to <code>logs/app-YYYY-MM-DD.log</code> on the <code>application-logs</code> share. Generated inventory reports are written to <code>exports/</code> on the same share.
Screenshots evidencing both services within the website	Section 6, figures 2 to 7.

SECTION 2

How work moves through the system

The queue is not a record of what already happened. It is what causes the work to happen, which is the distinction that makes the design worth anything under load.

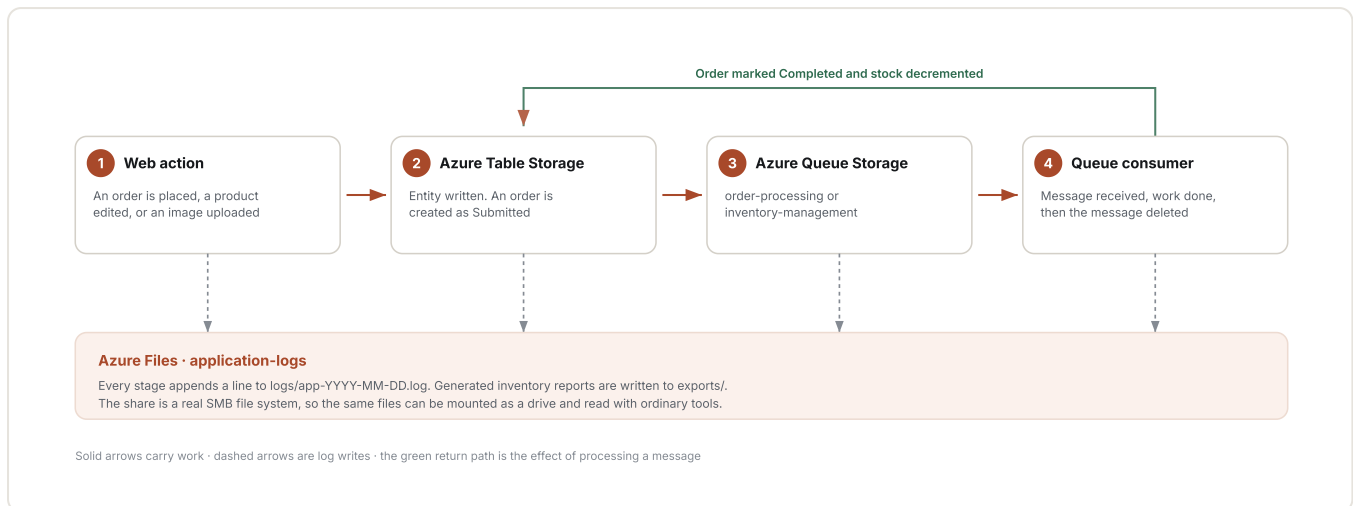


Figure 1. An action travels through Table Storage, onto a queue, and only takes effect once its message is consumed. Every stage writes to the file share.

Placing an order writes it to the Orders table as `Submitted` and queues a message. Nothing else happens yet. Processing that message is what marks the order complete and takes the stock off the product, which in turn queues a stock movement on the inventory queue. A burst of Christmas orders therefore accumulates on a queue that is designed to absorb it, rather than contending for the same database rows, which is exactly the failure the brief describes.

SECTION 3

Queue design

QUEUE	CARRIES
<code>order-processing</code>	Orders waiting to be picked, packed and dispatched
<code>inventory-management</code>	Catalogue and stock movements, including image uploads

Two queues rather than one, so that a backlog of catalogue edits cannot delay a customer's order. Each can also be given its own consumer, or scaled independently, without touching the other.

What a message carries

A queue message body is just a string. Each one therefore holds JSON with a readable description for the operator alongside the reference a consumer needs to act on it, so a single message serves both the monitoring screen and the processing code without a second lookup.

At-least-once delivery

Receiving a message does not remove it. Queue Storage hides it for a visibility timeout, and it is deleted only once the work has succeeded. A consumer that crashes part way through leaves the message to reappear and be retried, which is what makes the queue reliable rather than merely fast. The consequence is that a message can be delivered more than once, so the order handler checks whether an order is already complete before acting on it instead of assuming each message arrives exactly once.

Peek rather than receive for monitoring

The monitoring screen peeks at messages. Receiving them would start their visibility timeout and hide them from the code that is supposed to process them, so simply opening a dashboard would interfere with the workflow.

SECTION 4

Log files on Azure Files

Logs live on the `application-logs` file share rather than in a blob container. A share is a real SMB file system: it can be mounted as a drive on a server or a workstation and read with ordinary tools, which is what an operations team actually wants from a log. A blob would have to be downloaded first.

Appending, which Azure Files does not offer directly

Unlike an append blob, an Azure file has no append operation. Adding a line means reading the file's current length, growing the file by the size of the new bytes, and then writing those bytes into the range that growth created. That sequence is three calls and is not atomic, so writers are serialised

within the application. Across several App Service instances the correct answer would be a lease on the file; on the single instance this runs on, serialising is enough and is documented as such rather than left implicit.

Logging never breaks the request

A failure writing to the share is reported to the platform log and swallowed, and the user's action is allowed to complete. A customer losing their order because the log file was momentarily busy would plainly be the worse outcome.

SECTION 5

Implementation

COMPONENT	RESPONSIBILITY
<code>QueueStorageService</code>	Send, peek, dequeue, depth and clear over <code>Azure.Storage.Queues</code> .
<code>FileShareService</code>	Append, write, list and read over <code>Azure.Storage.Files.Shares</code> .
<code>ActivityRecorder</code>	Writes the queue message and the log line together.
<code>OrdersController</code>	Places orders and produces order-processing messages.
<code>QueuesController</code>	Monitors both queues and consumes messages off them.
<code>LogsController</code>	Browses the share, reads log files and writes inventory reports.

Controllers never call the queue and the file services separately. They call `IActivityRecorder`, which pairs the two writes, so no caller can queue work and forget to log it. The queue write comes first and is allowed to fail loudly: work that never reached the queue must not be reported as queued.

Every client is a singleton. `QueueClient` and `ShareClient` are thread safe and pool their connections, so one instance for the lifetime of the application avoids the socket exhaustion that follows from constructing a client per request. All four storage services sit behind interfaces resolved through dependency injection, which is why this task was additive rather than a rewrite of the Task 1 application.

SECTION 6

Screenshots

All captures are of the deployed application at <https://abcretail-gm-cldv7112.azurewebsites.net>, reading and writing the live `stabcretailgm001` storage account.

The screenshot displays the 'Queue monitor' interface for 'ABC.Retail'. The left sidebar contains a navigation menu with sections: OVERVIEW (Dashboard), TABLE STORAGE (Customers, Products), QUEUES AND FILES (Orders, Queue monitor, Log files), and REFERENCE (Storage Design). The main content area is titled 'Queue monitor' and includes a description: 'Order processing and inventory movements are written to Azure Queues rather than handled inline, so a burst of activity queues up instead of overwhelming the application. Messages below are peeked, not received, so opening this page does not hide them from the consumer.' A 'Place an order' button is in the top right. Below are two queue cards. The 'order-processing' card shows 1 waiting message: 'Processing order ORD-20260821-B11D for Michael Chen, 2 x Canvas Weekender Bag' with status 'OrderPlaced' and timestamp 'Queued 21 Aug 14:33:06'. The 'inventory-management' card shows 2 waiting messages: 'Stock adjusted for Aurora Wireless Headphones: 42 to 39 after ORD-20260821-FFD2' (status 'StockAdjusted', timestamp 'Queued 21 Aug 14:28:06') and 'Inventory report inventory-2026-08-21-142808.csv written to the application-logs file share' (status 'ReportGenerated', timestamp 'Queued 21 Aug 14:28:08'). Both cards have 'Process next message' and 'Clear queue' buttons. A section titled 'How a message moves' explains the at-least-once delivery process. The footer identifies the user as 'Genius Mhirizhonga' with role 'CLDV7112 - Cloud Development B' in 'Azure South Africa North'.

ABC.Retail
STORAGE CONSOLE

OVERVIEW
Dashboard

TABLE STORAGE
Customers
Products

QUEUES AND FILES
Orders
Queue monitor
Log files

REFERENCE
Storage Design

Genius Mhirizhonga
CLDV7112 - Cloud Development B
● Azure South Africa North

AZURE QUEUE STORAGE

Queue monitor

Order processing and inventory movements are written to Azure Queues rather than handled inline, so a burst of activity queues up instead of overwhelming the application. Messages below are peeked, not received, so opening this page does not hide them from the consumer.

Place an order

order-processing
Orders waiting to be picked, packed and dispatched
1 waiting

1

Processing order ORD-20260821-B11D for Michael Chen, 2 x Canvas Weekender Bag
OrderPlaced Queued 21 Aug 14:33:06

Process next message

Clear queue

inventory-management
Catalogue and stock movements, including image uploads
2 waiting

1

Stock adjusted for Aurora Wireless Headphones: 42 to 39 after ORD-20260821-FFD2
StockAdjusted Queued 21 Aug 14:28:06

2

Inventory report inventory-2026-08-21-142808.csv written to the application-logs file share
ReportGenerated Queued 21 Aug 14:28:08

Process next message

Clear queue

How a message moves

At-least-once delivery

Placing an order writes the entity to the Orders table and puts a message on `order-processing`. Processing that message is what completes the order and takes the stock off the product, which in turn queues a stock movement on `inventory-management`. Nothing advances until its message is handled.

Receiving a message hides it for a visibility timeout rather than removing it. It is deleted only once the work has succeeded, so a consumer that crashes mid-way leaves the message to reappear and be retried. That is at-least-once delivery, and it is why the order handler checks whether an order is already complete before acting on it.

Figure 2. Azure Queues in the website. Both queues with their live depth read from Azure. Messages are peeked rather than received, so opening this page does not hide them from the consumer. Processing the next message is what advances the work.

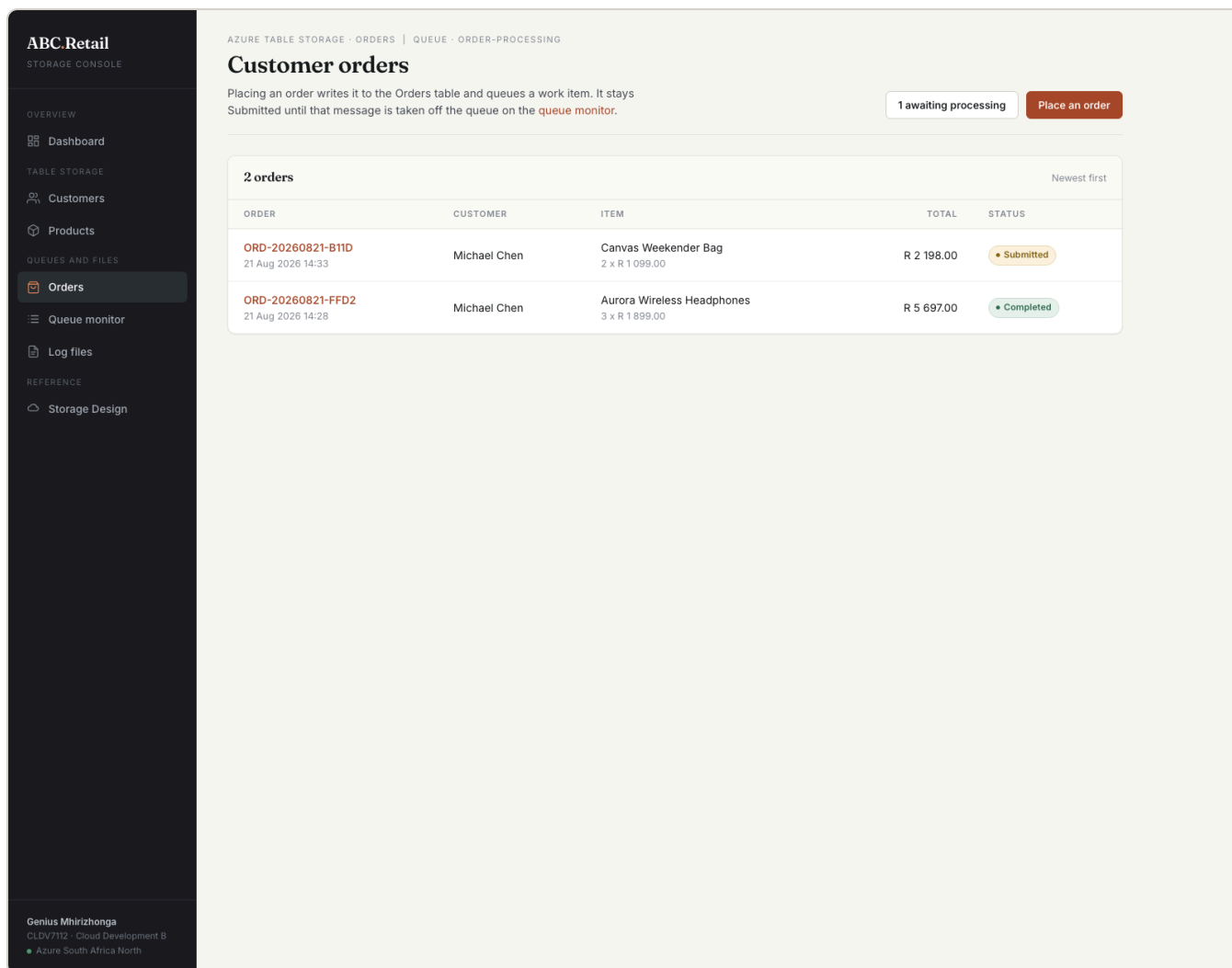


Figure 3. Orders waiting on the queue. An order stays Submitted until its message has been taken off order-processing. The status column is therefore a direct reflection of queue state.

ABC Retail

STORAGE CONSOLE

OVERVIEW

Dashboard

TABLE STORAGE

Customers

Products

QUEUES AND FILES

Orders

Queue monitor

Log files

REFERENCE

Storage Design

Genius Mhirizhonga

CLDV7712 - Cloud Development B

Azure South Africa North

ORDERS · NEW

Place an order

This writes the order to the Orders table, puts a message on the order-processing queue and appends a line to today's log file on the Azure file share.

Order detail

Stock is checked now, but only taken off the product once the queue message is processed.

Customer

Select a customer

Product

Select a product

Quantity

1

Submit and queue

Cancel

Figure 4. Producing a queue message. Submitting this form writes the order to the Orders table, puts a message on order-processing and appends a line to the log file on the file share.

ABC Retail

STORAGE CONSOLE

OVERVIEW

Dashboard

TABLE STORAGE

Customers

Products

QUEUES AND FILES

Orders

Queue monitor

Log files

REFERENCE

Storage Design

Genius Mhirizhonga

CLDV7712 - Cloud Development B

Azure South Africa North

AZURE FILES · APPLICATION-LOGS

Log files

Every action the application takes is appended to a daily log file on an Azure file share. A share is used rather than a blob because it is a real SMB file system: the operations team can mount it as a drive and read the logs with ordinary tools.

Write inventory report

logs1 file

app-2026-08-21.log737 B

exports1 file

inventory-2026-08-21-142808...834 B

SHARE

application-logs

STORED

1.5 KB across 2 files

logs/app-2026-08-21.logDownload

Newest entries first

2026-08-21 14:33:06INFOProcessing order ORD-20260821-B11D for Michael Chen, 2 x Canvas Weekender Bag [queued on order-processing]

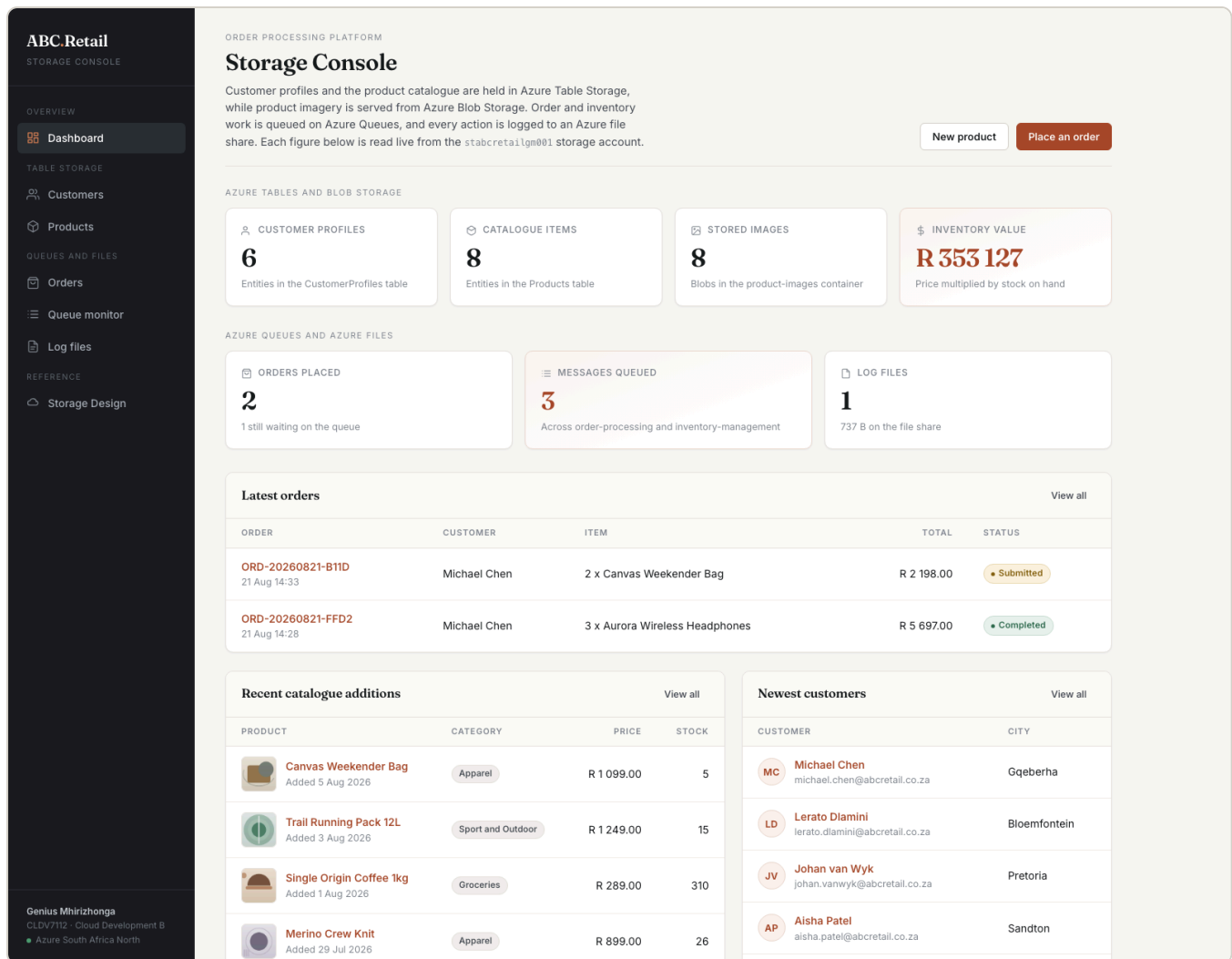
2026-08-21 14:28:08INFOInventory report inventory-2026-08-21-142808.csv written to the application-logs file share [queued on inventory-management]

2026-08-21 14:28:07INFOORD-20260821-FFD2 was processed and marked complete. Stock for Aurora Wireless Headphones moved from 42 to 39. [dequeued from order-processing]

2026-08-21 14:28:06INFOStock adjusted for Aurora Wireless Headphones: 42 to 39 after ORD-20260821-FFD2 [queued on inventory-management]

2026-08-21 14:28:05INFOProcessing order ORD-20260821-FFD2 for Michael Chen, 3 x Aurora Wireless Headphones [queued on order-processing]

Figure 5. Azure Files in the website. The application-logs share with its logs and exports directories, file sizes read from Azure, and the contents of the selected log file rendered in the browser.



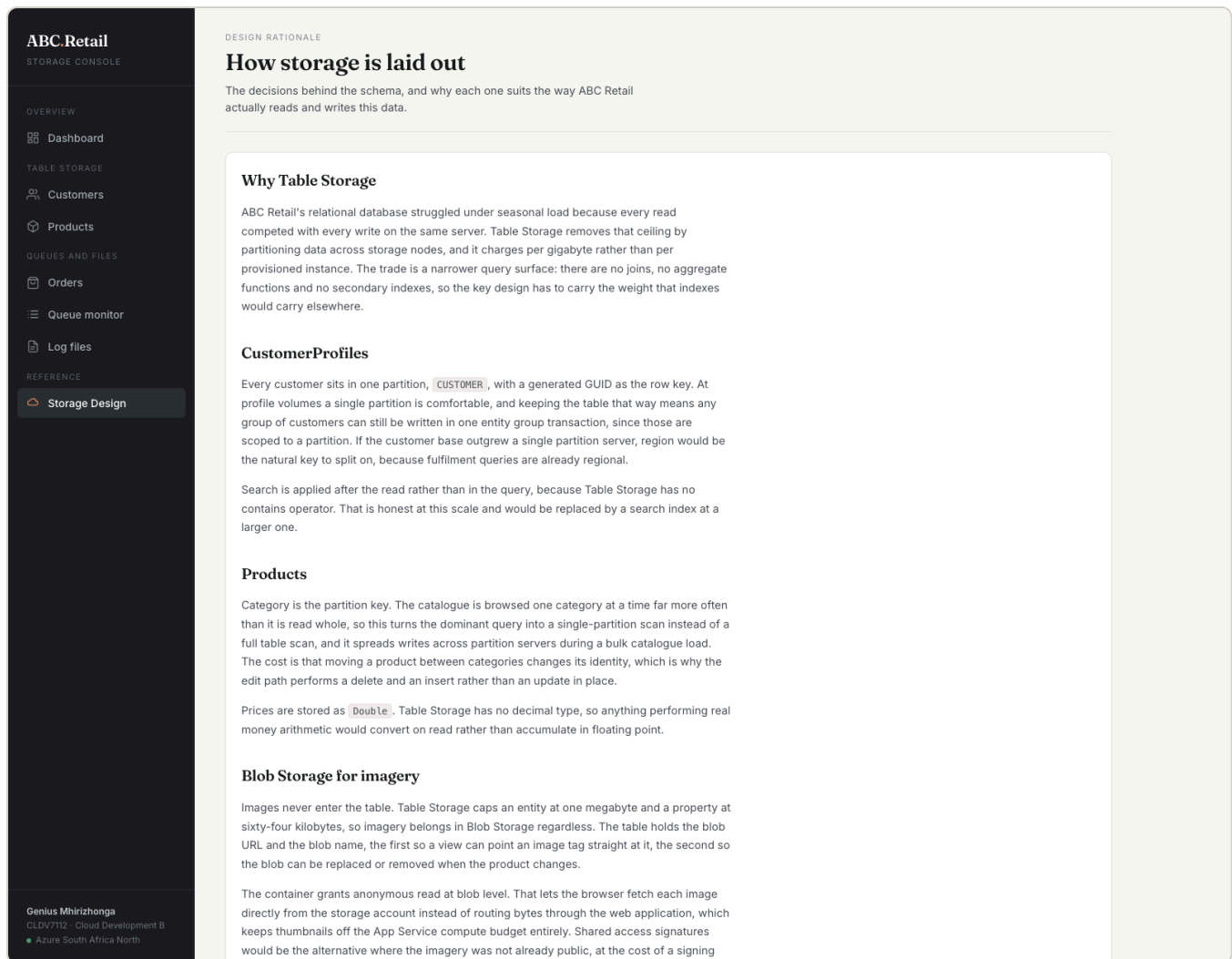


Figure 7. Design rationale published in the application. The queue and file share decisions are documented in the running site itself.

SECTION 7

Verification

The flow was exercised against the live storage account rather than assumed. Placing an order wrote the entity, raised the depth of `order-processing` and appended a log line. Processing that message marked the order complete, moved the stock for Aurora Wireless Headphones from 42 to 39, queued the resulting movement on `inventory-management` and logged both steps. Writing an inventory report produced a CSV in `exports/`. Each of those lines was then read back out of the share and rendered in the browser, which is what figure 5 shows.

The depth the site reports was checked against the storage account directly and matched: two messages on `inventory-management` and none on `order-processing` at the time of the check.